

מבוא למדעי המחשב

שיעור 3: עוד על סדרות ומיכלים אחרים

לקריאה נוספת:

- http://en.wikipedia.org/wiki/List_of_board_games
- <http://he.wikipedia.org/wiki/קפסאיצין>.
- http://en.wikipedia.org/wiki/Chili_pepper
- Madhur Jaffrey, *Indian Cookery*, BBC, 1984. (מהדורה עברית, 1994)
- Andreas Viestad and Mette Randem. *Where Flavor was Born: Recipes and Culinary Travels along the Indian Ocean Spice Route*, Chronicle Books, 2007.

dict-1 set

קבוצות

- הטיפוס set מאפשר להחזיק קבוצת איברים לא מסודרת.

- איתחול של קבוצה:
`our_set = {'tiddlywinks', 'whacamole'}`
`their_set = {'narro', 'mixedpickles'}`

- שייכות:
`'narro' in our_set`

- איחוד:
`s = our_set.union(their_set)`

- הוספת איבר:
`their_set.add('balderdash')`

- הסרת איבר:
`s.remove('mixedpickles')`

- יצירת קבוצה מסדרה:
`s = set("to be or not to be")`

דוגמת הרצה

Python 3.4.1 Shell

```
Python 3.4.1 (v3.4.1:c0e311e010fc, May 18 2014, 00:54:21)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "copyright", "credits" or "license()" for more information.
>>> our_set = {'tiddlywinks', 'whacamole'}
>>> our_set
{'tiddlywinks', 'whacamole'}
>>> their_set = {'narro', 'mixedpickles'}
>>> their_set
{'narro', 'mixedpickles'}
>>> 'narro' in our_set
False
>>> 'narro' in their_set
True
>>> s = our_set.union(their_set)
>>> s
{'narro', 'mixedpickles', 'tiddlywinks', 'whacamole'}
>>> their_set.add('balderdash')
>>> their_set
{'narro', 'balderdash', 'mixedpickles'}
>>> s.remove('mixedpickles')
>>> s
{'narro', 'tiddlywinks', 'whacamole'}
>>> our_set.remove('mixedpickles')
Traceback (most recent call last):
  File "<pyshell#12>", line 1, in <module>
    our_set.remove('mixedpickles')
KeyError: 'mixedpickles'
>>> s = set("to be or not to be")
>>> s
{'n', 't', 'e', 'b', ' ', 'o', 'r'}
>>>
```


עוד על set

- כל פריט חייב להיות immutable
- אובייקט מטיפוס set הוא mutable
frozenset הוא טיפוס דומה, רק immutable (ולכן אין לו פעולות שינוי)
- יש עוד הרבה פעולות (חיתוך, השוואה, ...).
- אפשר לעבור על כל האיברים ב-set בלולאת for (פייתון לא מתחייב לגבי הסדר). למשל,

```
for elem in our_set:  
    print(elem)
```

מדפיס את האיברים ב-our_set (בסדר כלשהו).

- מספר האיברים בקבוצה: `len(our_set)`



בעיית המילון

- רוצים להחזיק בזיכרון אוסף נתונים.
- כל פריט באוסף מורכב ממפתח (מילה) + ערך נלווה (פירוש המילה).
- צריך לאפשר את הפעולות הבאות:
 - הוספת פריט לאוסף: `insert(key, data)`
 - שליפת פריט על פי המפתח: `data = search(key)`
 - מחיקת פריט מהאוסף: `delete(key)`

מימוש

- בפייתון יש כלי נוח לממש מילון - אובייקט מטיפוס dict:

- אובייקט כזה דומה לרשימה, רק שהגישה לפריט היא על ידי המפתח שלו במקום האינדקס שלו.

```
our_dict = { }
```

- איתחול מילון ריק:

```
our_dict = {'zero' : 0, 'one' : 1}
```

- איתחול מילון עם ערכים:

```
our_dict[key] = value
```

- הוספת פריט:

```
our_dict['two'] = 2
```

לדוגמה:

```
data = our_dict[key]
```

- שליפת פריט:

```
digit = our_dict['zero']
```

לדוגמה:

```
del our_dict[key]
```

- מחיקת פריט:

```
del our_dict['two']
```

לדוגמה:

דוגמת הרצה

Python 3.4.1 Shell

```
Python 3.4.1 (v3.4.1:c0e311e010fc, May 18 2014, 00:54:21)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "copyright", "credits" or "license()" for more information.
>>> our_dict = {}
>>> our_dict
{}
>>> our_dict = {'zero' : 0, 'one' : 1}
>>> our_dict
{'one': 1, 'zero': 0}
>>> our_dict['two'] = 2
>>> our_dict
{'one': 1, 'two': 2, 'zero': 0}
>>> digit = our_dict['zero']
>>> digit
0
>>> del our_dict['two']
>>> our_dict
{'one': 1, 'zero': 0}
>>> del our_dict['two']
Traceback (most recent call last):
  File "<pyshell#10>", line 1, in <module>
    del our_dict['two']
KeyError: 'two'
>>> 'one' in our_dict
True
>>> for key in our_dict:
    print(key, " : ",our_dict[key])
```

דוגמת הרצה

```
Python 3.4.1 Shell
Python 3.4.1 (v3.4.1:c0e311e010fc, May 18 2014, 00:54:21)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "copyright", "credits" or "license()" for more information.
>>> our_dict = {}
>>> our_dict
{}
>>> our_dict = {'zero' : 0, 'one' : 1}
>>> our_dict
{'one': 1, 'zero': 0}
>>> our_dict['two'] = 2
>>> our_dict
{'one': 1, 'two': 2, 'zero': 0}
>>> digit = our_dict['zero']
>>> digit
0
>>> del our_dict['two']
>>> our_dict
{'one': 1, 'zero': 0}
>>> del our_dict['two']
Traceback (most recent call last):
  File "<pyshell#10>", line 1, in <module>
    del our_dict['two']
KeyError: 'two'
>>> 'one' in our_dict
True
>>> for key in our_dict:
    print(key, " : ",our_dict[key])
```

- המפתח של פריט חייב להיות immutable
- הערך יכול להיות mutable או immutable
- אובייקט dict מטיפוס mutable

עוד דוגמה

```
Python 3.4.1 Shell
Python 3.4.1 (v3.4.1:c0e311e010fc, May 18 2014, 00:54:21)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "copyright", "credits" or "license()" for more information.
>>> ingredients = {}
>>> ingredients['shakshuka'] = {'tomatos', 'eggs', 'garlic', 'green chilli', 'paprika',
    'white pepper', 'salt', 'vegetable oil'}
>>> ingredients['satay lembu'] = {'beef sirloin', 'cucumber', 'onion', 'lemongrass',
    'garlic', 'shallot', 'turmeric', 'galangal', 'cumin', 'tamarind paste', 'peanuts', 'salt',
    'peanut oil'}
>>> ingredients['til ke aloo'] = {'potatos', 'cumin seeds', 'black mustard seeds',
    'sesame seeds', 'Cayenne pepper', 'lemon', 'salt', 'vegetable oil'}
>>> print(ingredients['satay lembu'])
{'garlic', 'cucumber', 'onion', 'turmeric', 'lemongrass', 'salt', 'peanuts', 'sha
llot', 'galangal', 'peanut oil', 'cumin', 'tamarind paste', 'beef sirloin'}
>>> for key in ingredients:
    if 'vegetable oil' in ingredients[key]:
        print(key)

til ke aloo
shakshuka
>>>
```

comprehension

הדגמה

Python 3.4.1 Shell

```
Python 3.4.1 (v3.4.1:c0e311e010fc, May 18 2014, 00:54:21)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "copyright", "credits" or "license()" for more information.
>>> list_of_squares = [x**2 for x in range(10)]
>>> print(list_of_squares)
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
>>> dirty = 'Go ahead. Make my day.'
>>> print(dirty)
Go ahead. Make my day.
>>> clean_dirty = [dirty[i] for i in range(len(dirty))
                    if dirty[i] != ' ' or i == 0 or dirty[i-1] != ' ']
>>> print(clean_dirty)
['G', 'o', '.', 'a', 'h', 'e', 'a', 'd', '.', ' ', 'M', 'a', 'k', 'e', ' ', 'm',
'y', ' ', 'd', 'a', 'y', '.']
>>> clean_dirty_str = ''.join(dirty[i] for i in range(len(dirty))
                               if dirty[i] != ' ' or i == 0 or dirty[i-1] != ' ')
>>> print(clean_dirty_str)
Go ahead. Make my day.
>>> letters = {c for c in clean_dirty if c != ' '}
>>> print(letters)
{'y', 'o', '.', 'd', 'e', 'a', 'k', 'h', 'M', 'm', 'G'}
>>> frequency = {c: dirty.count(c) / len(dirty) for c in dirty if c != ' '}
>>> print(frequency)
{'m': 0.03333333333333333, 'M': 0.03333333333333333, 'k': 0.03333333333333333, 'o': 0.03333333333333333, '.': 0.06666666666666667, 'h': 0.03333333333333333, 'd': 0.06666666666666667, 'e': 0.06666666666666667, 'y': 0.06666666666666667, 'a': 0.13333333333333333, 'G': 0.03333333333333333}
>>>
```


הדגמה

Python 3.4.1 Shell

```
Python 3.4.1 (v3.4.1:c0e311e010fc, May 18 2014, 00:54:21)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "copyright", "credits" or "license()" for more information.
>>> list_of_squares = [x**2 for x in range(10)]
>>> print(list_of_squares)
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
>>> dirty = 'Go ahead. Make my day.'
>>> print(dirty)
Go ahead. Make my day.
>>> clean_dirty = [dirty[i] for i in range(len(dirty))
                    if dirty[i] != ' ' or i == 0 or dirty[i-1] != ' ']
>>> print(clean_dirty)
['G', 'o', '.', 'a', 'h', 'e', 'a', 'd', '.', ' ', 'M', 'a', 'k', 'e', ' ', ' ', 'm',
'y', ' ', 'd', 'a', 'y', '.']
>>> clean_dirty_str = ''.join(dirty[i] for i in range(len(dirty))
                              if dirty[i] != ' ' or i == 0 or dirty[i-1] != ' ')
>>> print(clean_dirty_str)
Go ahead. Make my day.
>>> letters = {c for c in clean_dirty if c != ' '}
>>> print(letters)
{'y', 'o', '.', 'd', 'e', 'a', 'k', 'h', 'M', 'm', 'G'}
>>> frequency = {c: dirty.count(c) / len(dirty) for c in dirty if c != ' '}
>>> print(frequency)
{'m': 0.03333333333333333, 'M': 0.03333333333333333, 'k': 0.03333333333333333, 'o': 0.03333333333333333, '.': 0.06666666666666667, 'h': 0.03333333333333333, 'd': 0.06666666666666667, 'e': 0.06666666666666667, 'y': 0.06666666666666667, 'a': 0.13333333333333333, 'G': 0.03333333333333333}
>>>
```

list comprehension
(מייצור list)

הדגמה

```
Python 3.4.1 Shell

Python 3.4.1 (v3.4.1:c0e311e010fc, May 18 2014, 00:54:21)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "copyright", "credits" or "license()" for more information.
>>> list_of_squares = [x**2 for x in range(10)]
>>> print(list_of_squares)
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
>>> dirty = 'Go ahead. Make my day.'
>>> print(dirty)
Go ahead. Make my day.
>>> clean_dirty = [dirty[i] for i in range(len(dirty))
                    if dirty[i] != ' ' or i == 0 or dirty[i-1] != ' ']
>>> print(clean_dirty)
['G', 'o', '.', 'a', 'h', 'e', 'a', 'd', '.', 'M', 'y', 'd', 'a', 'y', '.']
>>> clean_dirty_str = ''.join(dirty[i] for i in range(len(dirty))
                               if dirty[i] != ' ' or i == 0 or dirty[i-1] != ' ')
>>> print(clean_dirty_str)
Go ahead. Make my day.
>>> letters = {c for c in clean_dirty if c != ' '}
>>> print(letters)
{'y', 'o', '.', 'd', 'e', 'a', 'k', 'h', 'M', 'm', 'G'}
>>> frequency = {c: dirty.count(c) / len(dirty) for c in dirty if c != ' '}
>>> print(frequency)
{'m': 0.03333333333333333, 'M': 0.03333333333333333, 'k': 0.03333333333333333, 'o': 0.03333333333333333, '.': 0.06666666666666667, 'h': 0.03333333333333333, 'd': 0.06666666666666667, 'e': 0.06666666666666667, 'y': 0.06666666666666667, 'a': 0.1, 'G': 0.03333333333333333}
>>>
```

list comprehension
בתוספת פילטר

הדגמה

```
Python 3.4.1 Shell

Python 3.4.1 (v3.4.1:c0e311e010fc, May 18 2014, 00:54:21)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "copyright", "credits" or "license()" for more information.
>>> list_of_squares = [x**2 for x in range(10)]
>>> print(list_of_squares)
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
>>> dirty = 'Go ahead. Make my day.'
>>> print(dirty)
Go ahead. Make my day.
>>> clean_dirty = [dirty[i] for i in range(len(dirty))
                    if dirty[i] != ' ' or i == 0 or dirty[i-1] != ' ']
>>> print(clean_dirty)
['G', 'o', ' ', 'a', 'h', 'e', 'a', 'd', '.', ' ', 'M', 'a', 'k', 'e', ' ', ' ', 'm',
'y', ' ', 'd', 'a', 'y', '.']
>>> clean_dirty_str = ''.join(dirty[i] for i in range(len(dirty))
                              if dirty[i] != ' ' or i == 0 or dirty[i-1] != ' ')
>>> print(clean_dirty_str)
Go ahead. Make my day.
>>> letters = {c for c in clean_dirty if c != ' '}
>>> print(letters)
{'y', 'o', ' ', 'a', 'h', 'e', 'a', 'd', '.', ' ', 'M', 'a', 'k', 'e', ' ', ' ', 'm',
'y', ' ', 'd', 'a', 'y', '.'}
>>> freque
>>> print(
{'m': 0.03333333333333333, ' ': 0.03333333333333333, '.': 0.06666666666666667, 'h': 0.03333333333333333, 'd':
0.06666666666666667, 'e': 0.06666666666666667, 'y': 0.06666666666666667, 'a': 0.1
3333333333333333, 'G': 0.03333333333333333}
>>>
```

הקריאה *string.join(list)* משרשרת את סדרת המחרוזות,
מופרדות על ידי *string*.

הדגמה

Python 3.4.1 Shell

```
Python 3.4.1 (v3.4.1:c0e311e010fc, May 18 2014, 00:54:21)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "copyright", "credits" or "license()" for more information.
>>> list_of_squares = [x**2 for x in range(10)]
>>> print(list_of_squares)
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
>>> dirty = 'Go ahead. Make my day.'
>>> print(dirty)
Go ahead. Make my day.
>>> clean_dirty = [dirty[i] for i in range(len(dirty))
                    if dirty[i] != ' ' or i == 0 or dirty[i-1] != ' ']
>>> print(clean_dirty)
['G', 'o', '.', 'a', 'h', 'e', 'a', 'd', '.', ' ', 'M', 'a', 'k', 'e', ' ', ' ', 'm',
'y', ' ', 'd', 'a', 'y', '.']
>>> clean_dirty_str = ''.join(dirty[i] for i in range(len(dirty))
                               if dirty[i] != ' ' or i == 0 or dirty[i-1] != ' ')
>>> print(clean_dirty_str)
Go ahead. Make my day.
>>> letters = {c for c in clean_dirty if c != ' '}
>>> print(letters)
{'y', 'o', '.', 'd', 'e', 'a', 'k', 'h', 'M', 'm', 'G'}
>>> frequency = {c: dirty.count(c) / len(dirty) for c in letters}
>>> print(frequency)
{'m': 0.03333333333333333, 'M': 0.03333333333333333, '.': 0.06666666666666667, 'y': 0.06666666666666667, 'o': 0.06666666666666667, 'e': 0.06666666666666667, 'd': 0.06666666666666667, 'a': 0.1, 'G': 0.03333333333333333}
```

set comprehension
(set מייצור)

הדגמה

Python 3.4.1 Shell

```
Python 3.4.1 (v3.4.1:c0e311e010fc, May 18 2014, 00:54:21)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "copyright", "credits" or "license()" for more information.
>>> list_of_squares = [x**2 for x in range(10)]
>>> print(list_of_squares)
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
>>> dirty = 'Go ahead. Make my day.'
>>> print(dirty)
Go ahead. Make my day.
>>> clean_dirty = [dirty[i] for i in range(len(dirty))
                    if dirty[i] != ' ' or i == 0 or dirty[i-1] != ' ']
>>> print(clean_dirty)
['G', 'o', '.', 'a', 'h', 'e', 'a', 'd', '.', ' ', 'M', 'a', 'k', 'e', ' ', ' ', 'm',
'y', ' ', 'd', 'a', 'y', '.']
>>> clean_dirty_str = ''.join(dirty[i] for i in range(len(dirty))
                               if dirty[i] != ' ' or i == 0 or dirty[i-1] != ' ')
>>> print(clean_dirty_str)
Go ahead. Make my day.
>>> letters = {c for c in clean_dirty if c != ' '}
>>> print(letters)
{'y', 'o', '.', 'd', 'e', 'a', 'k', 'h', 'M', 'm', 'G'}
>>> frequency = {c: dirty.count(c) / len(dirty) for c in dirty if c != ' '}
>>> print(frequency)
{'m': 0.03333333333333333, 'M': 0.03333333333333333,
': 0.03333333333333333, '.': 0.06666666666666667, 'h':
0.06666666666666667, 'e': 0.06666666666666667, 'y': 0.
3333333333333333, 'G': 0.03333333333333333}
```

dict comprehension
(dict מייצר)

הדגמה

Python 3.4.1 Shell

```
Python 3.4.1 (v3.4.1:c0e311e010fc, May 18 2014, 00:54:21)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "copyright", "credits" or "license()" for more information.
>>> list_of_squares = [x**2 for x in range(10)]
>>> print(list_of_squares)
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
>>> dirty = 'Go ahead. Make my day.'
>>> print(dirty)
Go ahead. Make my day.
>>> clean_dirty = [dirty[i] for i in range(len(dirty))
                    if dirty[i] != ' ' or i == 0 or dirty[i-1] != ' ']
>>> print(clean_dirty)
['G', 'o', '.', 'a', 'h', 'e', 'a', 'd', '.', ' ', 'M', 'a', 'k', 'e', ' ', ' ', 'm',
'y', ' ', 'd', 'a', 'y']
>>> clean_dirty
>>> print(clean_dirty)
Go ahead. Make my day.

>>> letters = {c for c in clean_dirty if c != ' '}
>>> print(letters)
{'y', 'o', '.', 'd', 'e', 'a', 'k', 'h', 'M', 'm', 'G'}
>>> frequency = {c: dirty.count(c) / len(dirty) for c in dirty if c != ' '}
>>> print(frequency)
{'m': 0.03333333333333333, 'M': 0.03333333333333333, 'k': 0.03333333333333333, 'o': 0.03333333333333333, ' ': 0.06666666666666667, 'h': 0.03333333333333333, 'd': 0.06666666666666667, 'e': 0.06666666666666667, 'y': 0.06666666666666667, 'a': 0.13333333333333333, 'G': 0.03333333333333333}
>>>
```

הקריאה `string.count(char)` סופרת כמה פעמים מופיע `char` ב-`string`

הדגמה

Python 3.4.1 Shell

```
Python 3.4.1 (v3.4.1:c0e311e010fc, May 18 2014, 00:54:21)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "copyright", "credits" or "license()" for more information.
>>> list_of_squares = [x**2 for x in range(10)]
>>> print(list_of_squares)
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
>>> dirty = 'Go ahead. Make my day.'
>>> print(dirty)
Go ahead. Make my day.
>>> clean_dirty = [dirty[i] for i in range(len(dirty))
                    if dirty[i] != ' ' or i == 0 or dirty[i-1] != ' ']
>>> print(clean_dirty)
['G', 'o', '.', 'a', 'h', 'e', 'a', 'd', '.', ' ', 'M', 'a', 'k', 'e', ' ', 'm',
'y', ' ', 'd', 'a', 'y', '.']
>>> clean_dirty_str = ''.join(dirty[i] for i in range(len(dirty))
                               if dirty[i] != ' ' or i == 0 or dirty[i-1] != ' ')
>>> print(clean_dirty_str)
Go ahead. Make my day.
>>> letters = {c for c in clean_dirty if c != ' '}
>>> print(letters)
{'y', 'o', '.', 'd', 'e', 'a', 'k', 'h', 'M', 'm', 'G'}
>>> frequency = {c: dirty.count(c) / len(dirty) for c in dirty if c != ' '}
>>> print(frequency)
{'m': 0.03333333333333333, 'M': 0.03333333333333333, 'k': 0.03333333333333333, 'o': 0.03333333333333333, '.': 0.06666666666666667, 'h': 0.03333333333333333, 'd': 0.06666666666666667, 'e': 0.06666666666666667, 'y': 0.06666666666666667, 'a': 0.13333333333333333, 'G': 0.03333333333333333}
>>>
```

עוד הדגמה

Python 3.4.1: morecomprehension.py - /Users/yrabani/Dropbox/... א. מיכלים - סדרות ושיעור 3 - תשעז/תשעז/פייתון/המחשב - הוראה/מבוא למדעי המחשב

```
def divisors(n):
    """ returns a list of the proper divisors of n
        the input is assumed to be a positive int
        the output is a list of ints """
    return [i for i in range(1,n) if n%i == 0]

def prime(n):
    """ returns True iff n is a prime number
        n is assumed to be a positive int """
    return divisors(n) == [1]

def perfect(n):
    """ returns True iff n is a perfect number
        i.e., n equals the sum of its proper divisors
        n is assumed to be a positive int """
    return sum(divisors(n)) == n

def primes(n):
    """ returns a list of the primes up to n
        n is assumed to be a positive int """
    return [i for i in range(2,n+1) if prime(i)]

def perfect_numbers(n):
    """ returns a list of perfect numbers up to n
        n is assumed to be a positive int """
    return [i for i in range(2,n+1) if perfect(i)]
```


עוד הדגמה

Python 3.4.1: morecomprehension.py - /Users/yrabani/Dropbox/... א - סדרות ומיכלים - 3 - שיעור/תשעז/שיעור 3 - הוראה/מבוא למדעי המחשב - פייתון/תשעז/שיעור 3 - סדרות ומיכלים א

```
def divisors(n):  
    """ returns a list of the proper divisors of n  
        the input is assumed to be a positive int
```

Python 3.4.1 Shell

Python 3.4.1 (v3.4.1:c0e311e010fc, May 18 2014, 00:54:21)

[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin

Type "copyright", "credits" or "license()" for more information.

```
>>> ===== RESTART =====
```

```
>>>
```

```
>>> primes(30)
```

```
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
```

```
>>> perfect_numbers(10000)
```

```
[6, 28, 496, 8128]
```

```
>>>
```

Ln: 10 Col: 4

```
def primes(n):  
    """ returns a list of the primes up to n  
        n is assumed to be a positive int """  
    return [i for i in range(2,n+1) if prime(i)]
```

```
def perfect_numbers(n):  
    """ returns a list of perfect numbers up to n  
        n is assumed to be a positive int """  
    return [i for i in range(2,n+1) if perfect(i)]
```

ביטויי למדא

ביטויי למדא

- זה ביטוי מהצורה: `lambda parameter_list : expression`
שבו `parameter_list` היא רשימת משתנים מופרדים בפסיקים
ו-`expression` הוא ביטוי המשתמש במשתנים הללו.

- הערך שלו הוא פונקציה שמחשבת את `expression`.

- למשל, הפקודה
`prod = lambda x, y : x*y`
שקולה לקטע הקוד הבא:

```
def prod(x,y):  
    return x*y
```

ביטויי למדא

- זה ביטוי מהצורה: `lambda parameter_list : expression`
שבו `parameter_list` היא רשימת משתנים מופרדים בפסיקים

למ(ב)דא (lambda) היא האות היוונית λ שמקבילה לאות למד בעברית (גם השמות מאותו מקור).

אין לבלבל אותה עם ריקוד הלמבדה (lambada) שאולי מוכר לכם מסרטים או מברזיל.

```
def prod(x,y):  
    return x*y
```

פונקציות מסדר שני

- החשיבות של ביטויי למדא מתבהרת כאשר רוצים להגדיר פונקציה מסדר שני (כלומר, פונקציה שמחזירה פונקציה).

- לדוגמה, הנה פונקציה שמקבלת כקלט פונקציה אחרת `func` ומוציאה כפלט פונקציה שהיא הנגזרת (המקורבת) של `func`. הפרמטר השני `eps` שולט במידת הדיוק של הפלט:

```
def derivative(func, eps):  
    return lambda x: (func(x+eps) - func(x)) / eps
```

- לדוגמה, הביטוי `derivative(lambda x: x**2, 0.001)(5)` מחשב את הנגזרת של x^2 בנקודה $x = 5$.

- שימו לב: הביטוי הזה מפעיל את הפלט של `derivative` (שזו פונקציה) על הפרמטר 5.

הערות

- בפייתון, פונקציה היא אובייקט מטיפוס `function`.
- אז פונקציה היא גם קוד שניתן להרצה וגם אובייקט שמחזיק נתונים (`data`), כמו `int` או `list` או דוגמאות אחרות שראינו.
- פונקציה יכולה לקבל פונקציה כפרמטר וגם להחזיר פונקציה כפלט.
- בעזרת ביטויי למדא אפשר לייצר פונקציות תוך כדי ריצה.
- בהמשך נרחיב על זה מעט יותר.

עיבוד תמונה

עיבוד אותות ספרתי

- מדידה, ייצוג ומניפולציה של אותות מידע ממקורות שונים (קול, תמונה, מכ"ם וסונאר, חיישנים, ...)
- אנחנו נתרכז במעט טכניקות פשוטות הקשורות בעיבוד תמונה.
- מה נראה? -

– ייצוג תמונה (שחור-לבן ובצבע)

– פילטרים פשוטים

– היסטוגרמות ופילטרים שמשתמשים בהן

הספרייה Pillow

- זו ספרייה הכוללת יכולות בסיסיות רבות של עיבוד תמונה. מבוססת על גרסה קודמת שנקראה PIL (ראשי תיבות של Python Imaging Library).
- בתרגול תראו בין היתר שימוש ב-Pillow. בשעור נעשה הכל לבד, למעט קריאה של קבצי תמונות והצגה על המסך של תמונות.
- מידע והדרכה על Pillow ניתן למצוא כאן:
<https://pillow.readthedocs.org/en/latest/>

ייצוג תמונה

- פיסית, מסך של מחשב (או טלויזיה) הוא מערך דו-מימדי של פיקסלים.
- פיקסל (pixel) משמעו picture element.
- כל פיקסל הוא תחום זעירי שאפשר להאיר בעוצמה משתנה.
- אם הנקודות צפופות דיין, מתקבל הרושם של תמונה רציפה.
- במסכי צבע כל פיקסל מורכב משלוש נקודות אור בצבעים אדום, ירוק, כחול (RGB).
- קומבינציה של עוצמות של הצבעים הללו מייצגת גוון ספציפי.

ייצוג תמונה בזיכרון

- הייצוג בזיכרון תואם למבנה הפיסי של מסך - מערך דו-מימדי של פיקסלים.
- כל פיקסל הוא שלשה של מספרים שמציינים את העוצמה של אדום, ירוק, כחול.
- ערך העוצמה הוא בדרך כלל מספר שלם בין 0 (כבוי) ל-255 (עוצמה מירבית).
- בתמונות שחור-לבן (יש עדיין שימושים שלא צריכים צבע), כל פיקסל הוא מספר בודד.

ייצוג תמונה בקובץ

- כאשר שומרים תמונה בקובץ, בדרך כלל מעבירים אותה לפורמט דחוס.
- יש מגוון של תקנים של קבצי תמונות, חלקם בדחיסה משמרת מידע (למשל GIF או PNG), חלקם בדחיסה מאבדת מידע (למשל JPEG).
- על מנת לעבד ולהציג את התמונה יש לשחזר את הייצוג כמערך דו-מימדי של פיקסלים.

השימוש המזערי שנעשה ב-Pillow

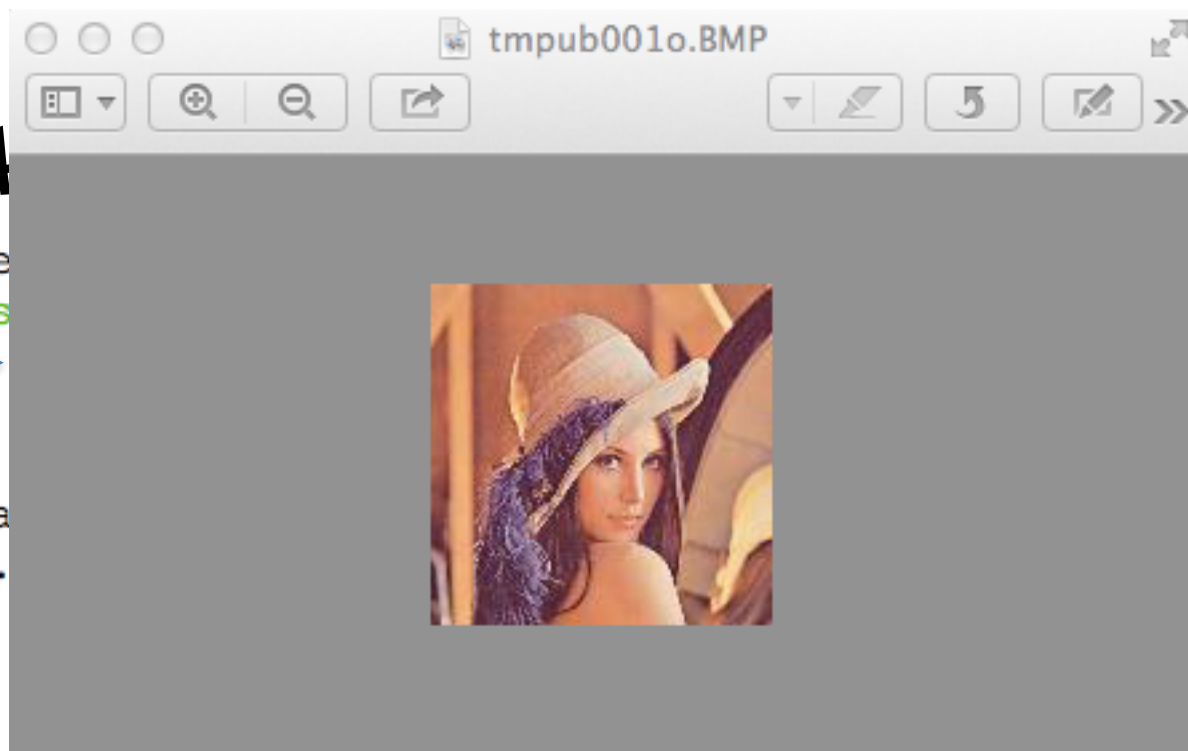
```
>>> from PIL import Image
>>> img = Image.open("/Users/yrabani/lena.jpg")
>>> img.show()
>>> img.size
(128, 128)
>>> px = list(img.getdata())
>>> ar = two_dim(px, img.size[1], img.size[0])
>>> px[0]
(226, 161, 119)
>>> px[129]
(225, 161, 117)
>>> ar[0][0]
(226, 161, 119)
>>> ar[1][1]
(225, 161, 117)
>>> for i in range(len(ar)):
        ar[i][0] = (255, 255, 255)

>>> new_px = one_dim(ar)
>>> img.putdata(new_px)
>>> img.show()
>>>
```

שה ב-Pillow

```
>>> from PIL import Image
>>> img = Image.open("/Us
>>> img.show() →
>>> img.size
(128, 128)
>>> px = list(img.getdata
>>> ar = two_dim(px, img.
>>> px[0]
(226, 161, 119)
>>> px[129]
(225, 161, 117)
>>> ar[0][0]
(226, 161, 119)
>>> ar[1][1]
(225, 161, 117)
>>> for i in range(len(ar)):
        ar[i][0] = (255, 255, 255)

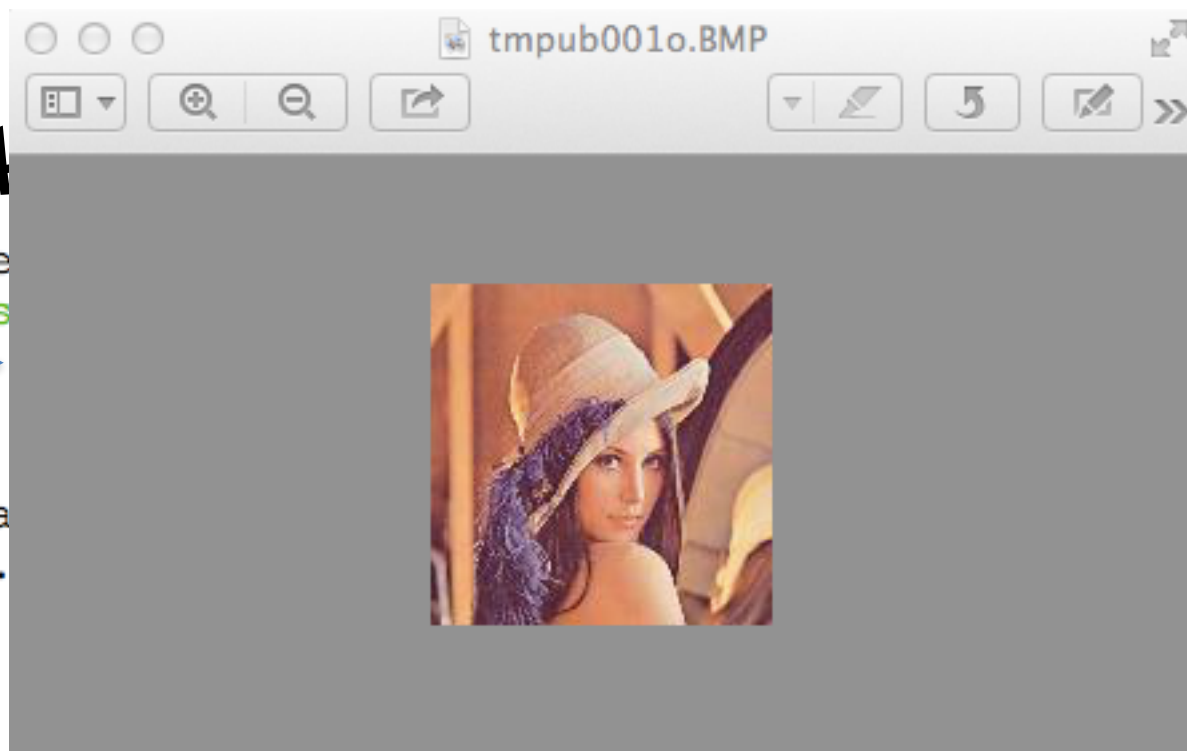
>>> new_px = one_dim(ar)
>>> img.putdata(new_px)
>>> img.show()
>>>
```



שה ב-Pillow

```
>>> from PIL import Image
>>> img = Image.open("/Us
>>> img.show()
>>> img.size
(128, 128)
>>> px = list(img.getdata
>>> ar = two_dim(px, img.
>>> px[0]
(226, 161, 119)
>>> px[129]
(225, 161, 117)
>>> ar[0][0]
(226, 161, 119)
>>> ar[1][1]
(225, 161, 117)
>>> for i in range(len(ar)):
    ar[i][0] = (255, 255, 255)

>>> new_px = one_dim(ar)
>>> img.putdata(new_px)
>>> img.show()
>>>
```



לנה סודרברג, דף האמצע של פלייבוי, נובמבר 1972

השימוש המזערי שנעשה ב-Pillow

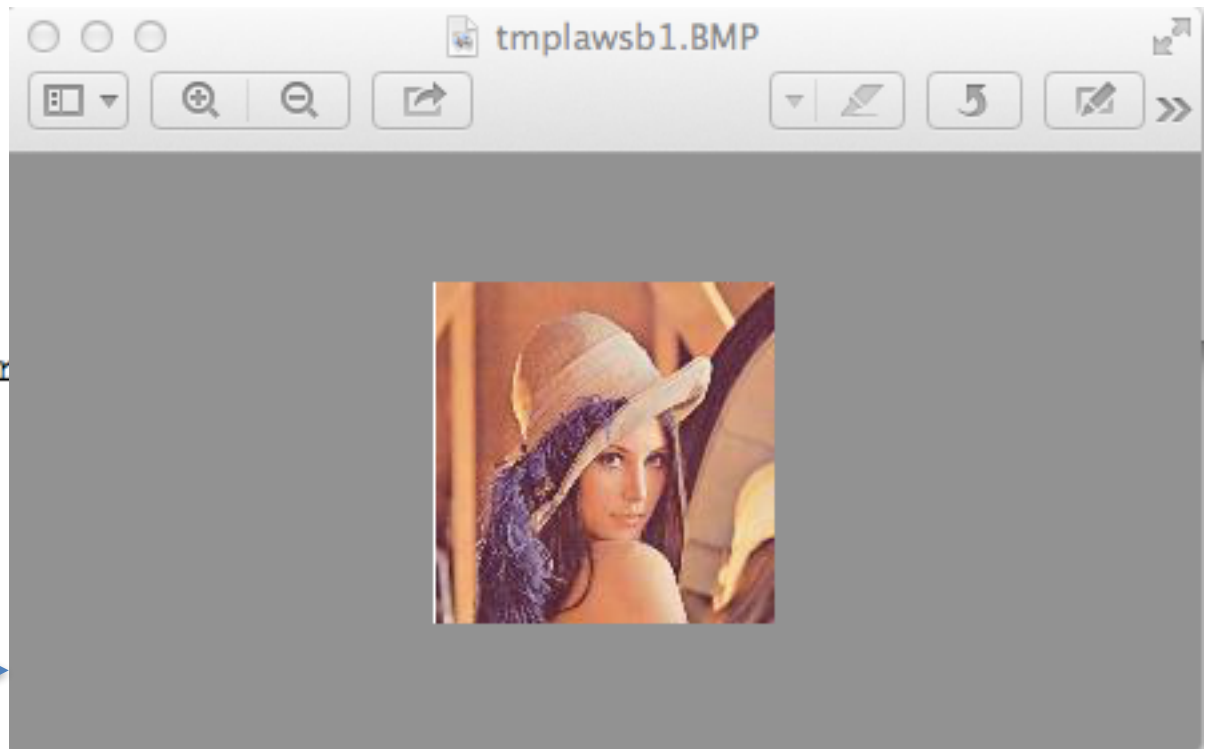
```
>>> from PIL import Image
>>> img = Image.open("/Users/yrabani/lena.jpg")
>>> img.show()
>>> img.size
(128, 128)
>>> px = list(img.getdata())
>>> ar = two_dim(px, img.size[1], img.size[0])
>>> px[0]
(226, 161, 119)
>>> px[129]
(225, 161, 117)
>>> ar[0][0]
(226, 161, 119)
>>> ar[1][1]
(225, 161, 117)
>>> for i in range(len(ar)):
        ar[i][0] = (255, 255, 255)

>>> new_px = one_dim(ar)
>>> img.putdata(new_px)
>>> img.show()
>>>
```


השימוש המזערי שנעשה ב-Pillow

```
>>> from PIL import Image
>>> img = Image.open("/Users/yrabani/lena.jpg")
>>> img.show()
>>> img.size
(128, 128)
>>> px = list(img.getdata())
>>> ar = two_dim(px, img.size[1], img.size[0])
>>> px[0]
(226, 161, 119)
>>> px[129]
(225, 161, 117)
>>> ar[0][0]
(226, 161, 119)
>>> ar[1][1]
(225, 161, 117)
>>> for i in range(len(ar)
        ar[i][0] = (255,

>>> new_px = one_dim(ar)
>>> img.putdata(new_px)
>>> img.show()
>>>
```



פונקציות העזר

```
>>> def two_dim(px, nrows, ncols):  
    if len(px) != nrows * ncols:  
        return None  
    else:  
        return [[px[ncols * i + j] for j in range(ncols)] for i in range(nrows)]  
  
>>> def one_dim(ar):  
    return [e for row in ar for e in row]  
  
>>>
```

התמרות פשוטות

- נשנה את התמונה על ידי החלפת שלשת הערכים של פיקסל בערך פונקציה f על השלשה הזו.
- למשל: תמונת נגטיב מתקבלת מההמרה הבאה: $(r, g, b) \rightarrow (255-r, 255-g, 255-b)$

קצת קוד

```
>>> def get_image(fname):
    img = Image.open(fname)
    px = list(img.getdata())
    ar = two_dim(px, img.size[1], img.size[0])
    return img, ar

>>> def put_image(img, ar):
    new_img = img.copy()
    px = one_dim(ar)
    new_img.putdata(px)
    return new_img

>>> def simple_transform(ar, f):
    return [[f(ar[i][j]) for j in range(len(ar[i]))] for i in range(len(ar))]

>>>
```

דוגמאות

```
>>> (img, ar) = get_image("/Users/yrabani/IMG_3552.jpg")
>>> img.show()
>>> new_ar = simple_transform(ar, lambda pix: tuple(255-x for x in pix))
>>> new_img = put_image(img, new_ar)
>>> new_img.show()
>>> (img, ar) = get_image("/Users/yrabani/IMG_1730.jpg")
>>> img.show()
>>> new_ar = simple_transform(ar, lambda pix: tuple(int(255*(x/255)**2) for x in pix))
>>> new_img = put_image(img, new_ar)
>>> img.show()
>>> new_img.show()
>>>
```

```
>>> (img, ar) = get_image("/
>>> img.show()
>>> new_ar = simple_transfor
>>> new_img = put_image(img,
>>> new_img.show()
>>> (img, ar) = get_image("/
>>> img.show()
>>> new_ar = simple_transfor
>>> new_img = put_image(img,
>>> img.show()
>>> new_img.show()
>>>
```



```
r x in pix))
```

דוגמאות

```
>>> (img, ar) = get_image("/Users/yrabani/IMG_3552.jpg")
>>> img.show()
>>> new_ar = simple_transform(ar, lambda pix: tuple(255-x for x in pix))
>>> new_img = put_image(img, new_ar)
>>> new_img.show()
>>> (img, ar) = get_image("/Users/yrabani/IMG_1730.jpg")
>>> img.show()
>>> new_ar = simple_transform(ar, lambda pix: tuple(int(255*(x/255)**2) for x in pix))
>>> new_img = put_image(img, new_ar)
>>> img.show()
>>> new_img.show()
>>>
```

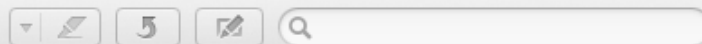
```
>>> (img, ar) = get_image("/User
>>> img.show()
>>> new_ar = simple_transform(ar
>>> new_img = put_image(img, new
>>> new_img.show()
>>> (img, ar) = get_image("/User
>>> img.show()
>>> new_ar = simple_transform(ar
>>> new_img = put_image(img, new
>>> img.show()
>>> new_img.show()
>>>
```



in pix))

דוגמאות

```
>>> (img, ar) = get_image("/Users/yrabani/IMG_3552.jpg")
>>> img.show()
>>> new_ar = simple_transform(ar, lambda pix: tuple(255-x for x in pix))
>>> new_img = put_image(img, new_ar)
>>> new_img.show()
>>> (img, ar) = get_image("/Users/yrabani/IMG_1730.jpg")
>>> img.show()
>>> new_ar = simple_transform(ar, lambda pix: tuple(int(255*(x/255)**2) for x in pix))
>>> new_img = put_image(img, new_ar)
>>> img.show()
>>> new_img.show()
>>>
```



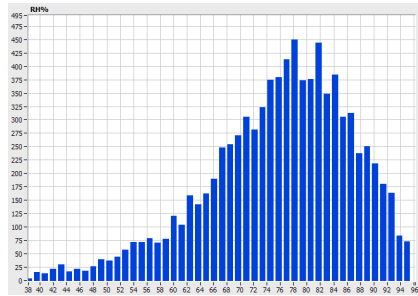
דוגמאות

```
>>> (img, ar) = get_image("/Users/yrabani/IMG_3552.jpg")
>>> img.show()
>>> new_ar = simple_transform(ar, lambda pix: tuple(255-x for x in pix))
>>> new_img = put_image(img, new_ar)
>>> new_img.show()
>>> (img, ar) = get_image("/Users/yrabani/IMG_1730.jpg")
>>> img.show()
>>> new_ar = simple_transform(ar, lambda pix: tuple(int(255*(x/255)**2) for x in pix))
>>> new_img = put_image(img, new_ar)
>>> img.show()
>>> new_img.show()
>>>
```




דוגמאות

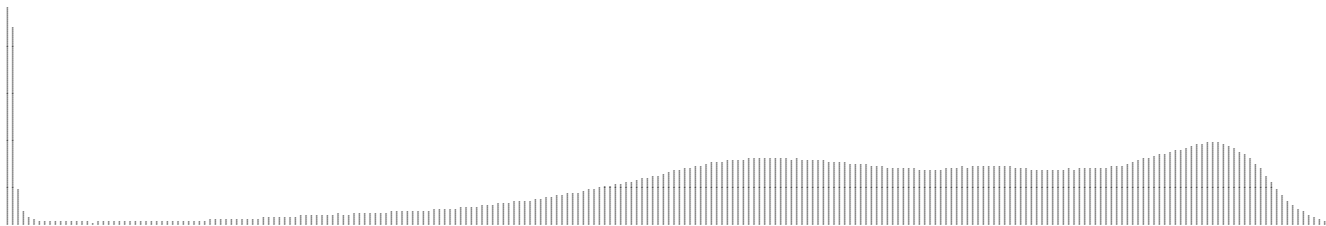
```
>>> (img, ar) = get_image("/Users/yrabani/IMG_3552.jpg")
>>> img.show()
>>> new_ar = simple_transform(ar, lambda pix: tuple(255-x for x in pix))
>>> new_img = put_image(img, new_ar)
>>> new_img.show()
>>> (img, ar) = get_image("/Users/yrabani/IMG_1730.jpg")
>>> img.show()
>>> new_ar = simple_transform(ar, lambda pix: tuple(int(255*(x/255)**2) for x in pix))
>>> new_img = put_image(img, new_ar)
>>> img.show()
>>> new_img.show()
>>>
```



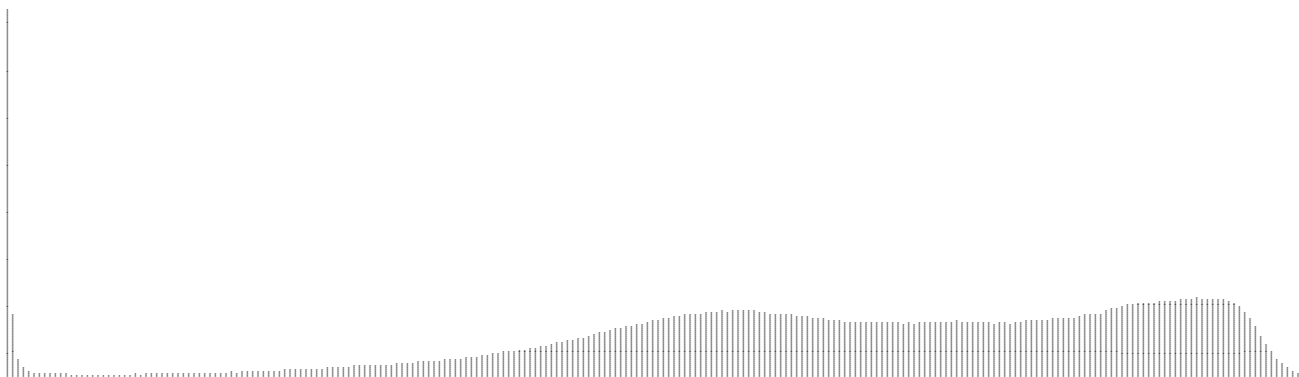
חישוב היסטוגרמה

- ההיסטוגרמה מציינת לכל ערך בין 0 ל-255 כמה פיקסלים יש עם ערך זה.
- עבור תמונה צבעונית נחשב בנפרד שלוש היסטוגרמות לכל אחד מהצבעים.
- לרוב נרצה לנרמל לאחוזים או הסתברויות במקום מונים מוחלטים.

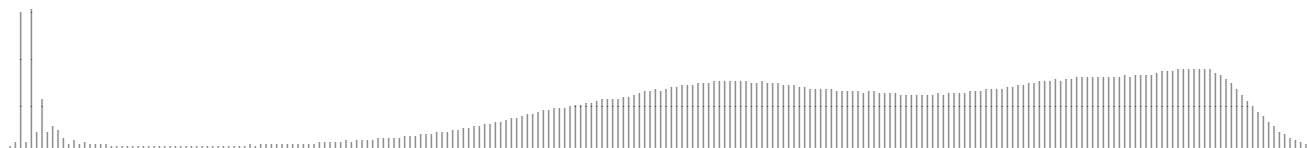
דוגמה



אדום



ירוק



כחול

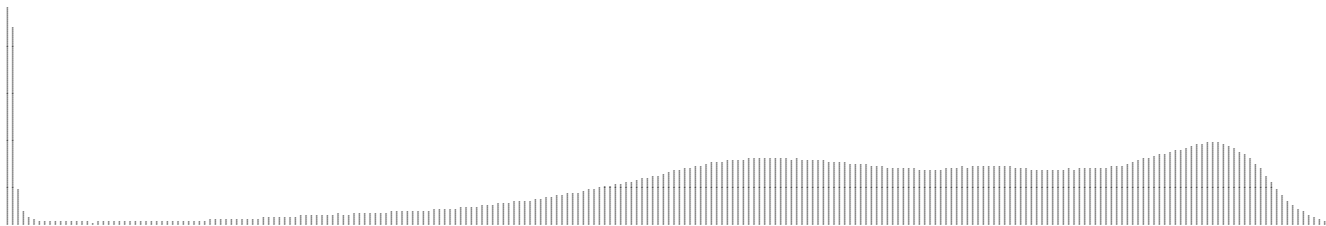


אדום

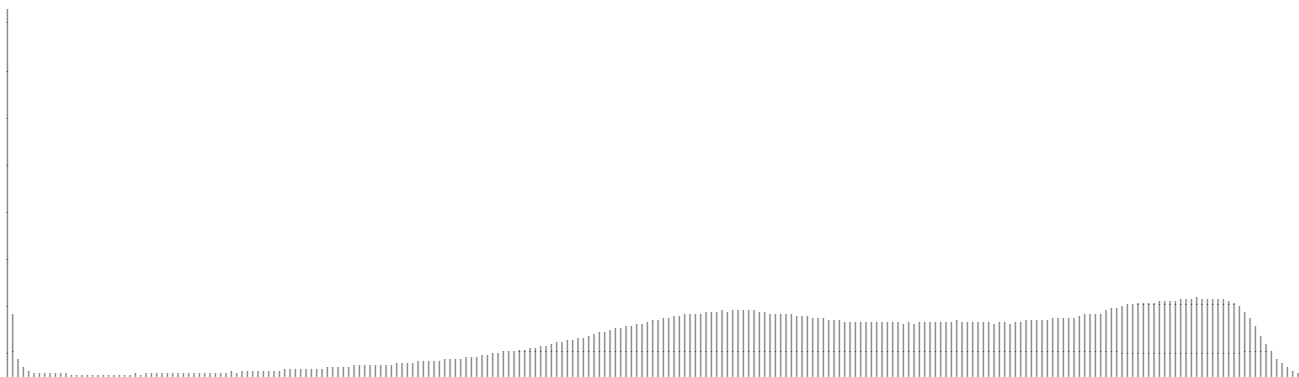
ירוק

כחול

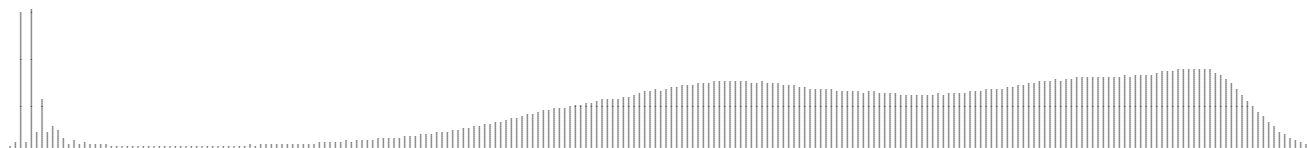
דוגמה



אדום



ירוק



כחול

שיטוח ההיסטוגרמה

- ההתמרה משנה את הגוונים כך שתיווצר היסטוגרמה "שטוחה" ככל האפשר.
- פיקסלים עם אותו גוון נשארים עם אותו גוון.
- נסמן ב- $h(p)$ את שיעור הפיקסלים עם ערך p (עבור צבע ספציפי).
- נסמן ב- $cdf(p)$ את ההסתברות המצרפית, כלומר, $h(0) + h(1) + \dots + h(p)$.
- הערך החדש של פיקסל שערכו הישן p הוא $int(255 * cdf(p))$.

עוד קוד

[illegible]

קוד

```

def simple_transform(pixels, f):
    """ applies f to each of the pixels.
        assumptions:
            pixels is a list of lists (2-dim)
            len(pixels[i]) is the same for all i
            each pixel is an RGB triple (len == 3)
            the returned value has the same structure
    """
    return [[f(pixels[i][j]) for j in range(3)] for i in range(len(pixels))]

def histogram(pixels, rgb, nvals = 256):
    """ computes the red or green or blue histogram
        ... """
    hist = [0] * nvals
    for row in pixels:
        for pixel in row:
            hist[pixel[rgb]] += 1
    return hist

def histeq_transform(pixels):
    """ transforms the image to spread histogram
        as possible ... """
    cdf = 3 * [None] # for each color c, 256 values
                    # new intensity of pixel
    for rgb in range(3):
        hist = histogram(pixels, rgb)
        dist = [h / sum(hist) for h in hist]
        cdf[rgb] = [sum(dist[i] for i in range(j)) for j in range(256)]
    return simple_transform(pixels, lambda p: [p[j] * cdf[rgb][p[j]] for j in range(3)])

```



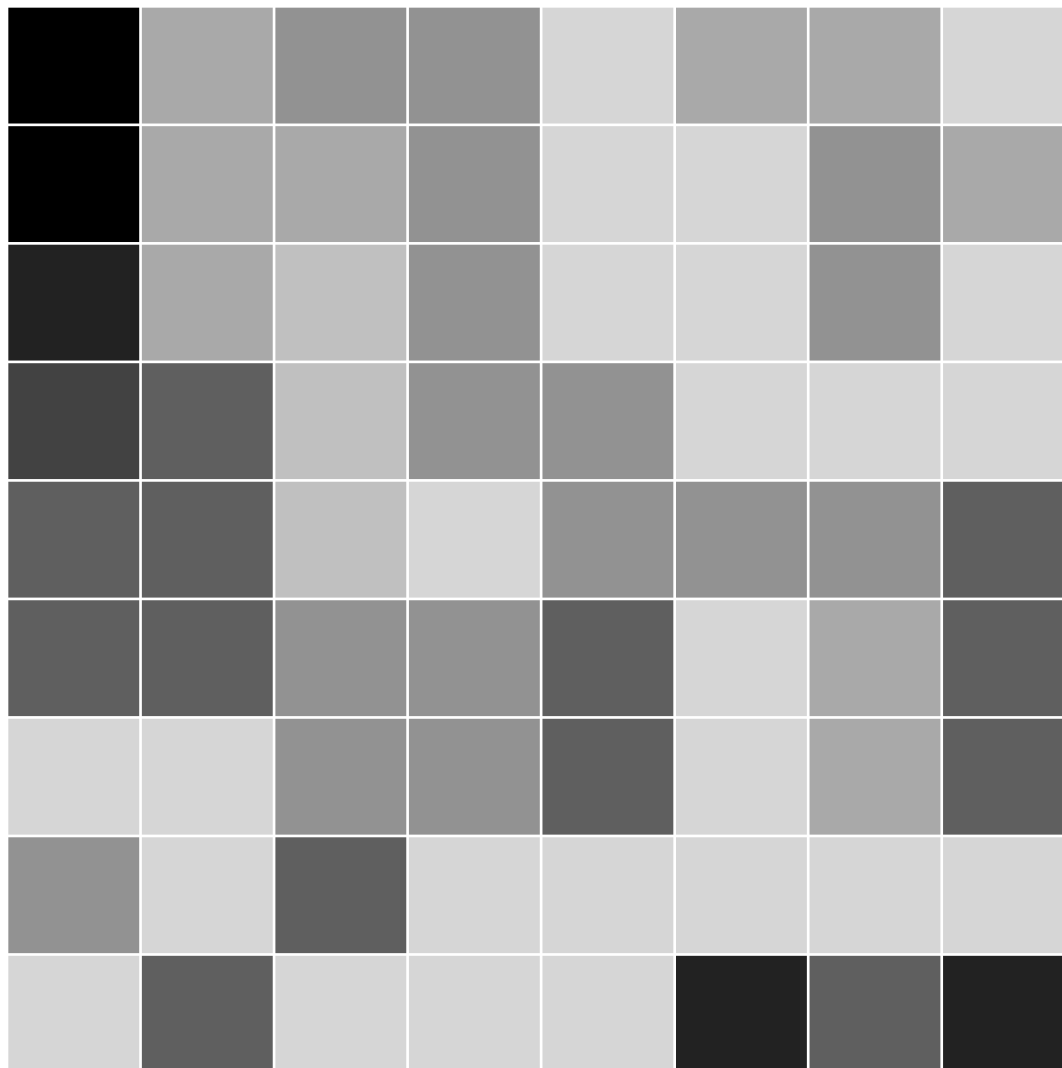




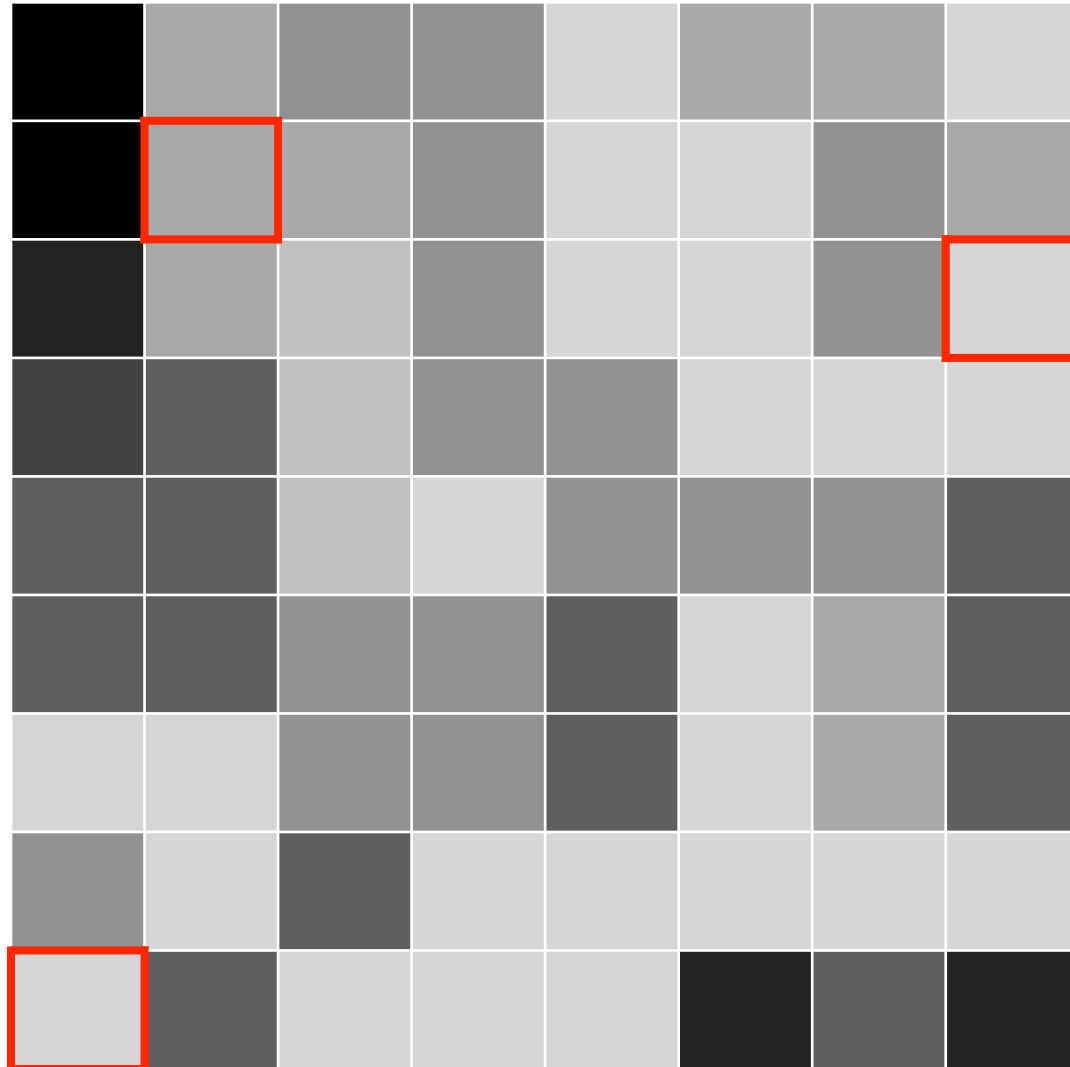
טשטוש תמונה

- על מנת לטשטש, נחליף את הערך של כל פיקסל בממוצע הערכים של הפיקסלים בסביבתו.
- אנחנו ניקח ריבוע של 3×3 שמרכזו בפיקסל ונחליף את ערך הפיקסל (עבור כל צבע) בממוצע הערכים בריבוע.
- הפעלה חוזרת של ההתמרה מטשטשת יותר.
- יש לשים לב לקצות התמונה, שם הריבוע שלנו חורג מגבולות התמונה.

טשטוש תמונה



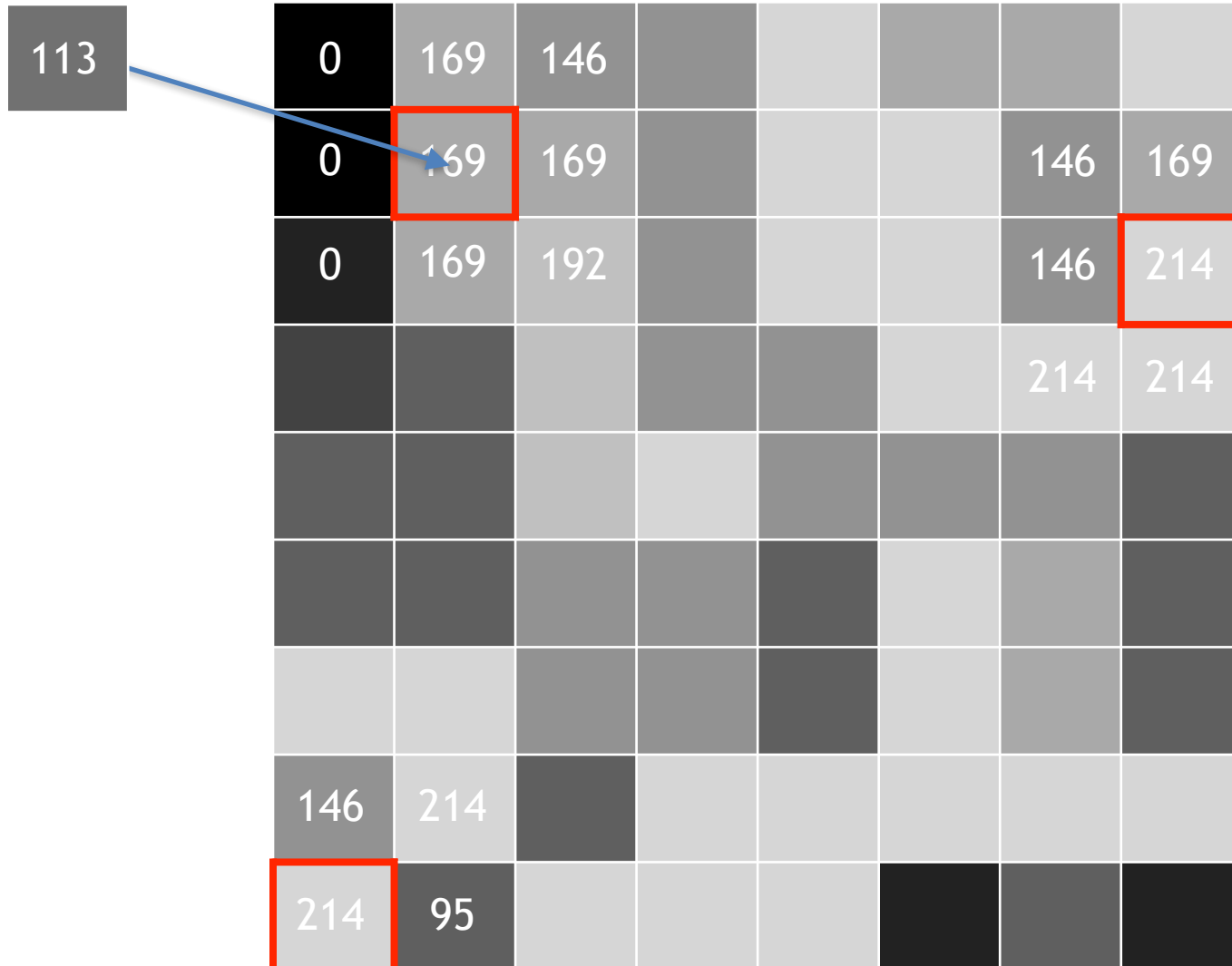
טשטוש תמונה



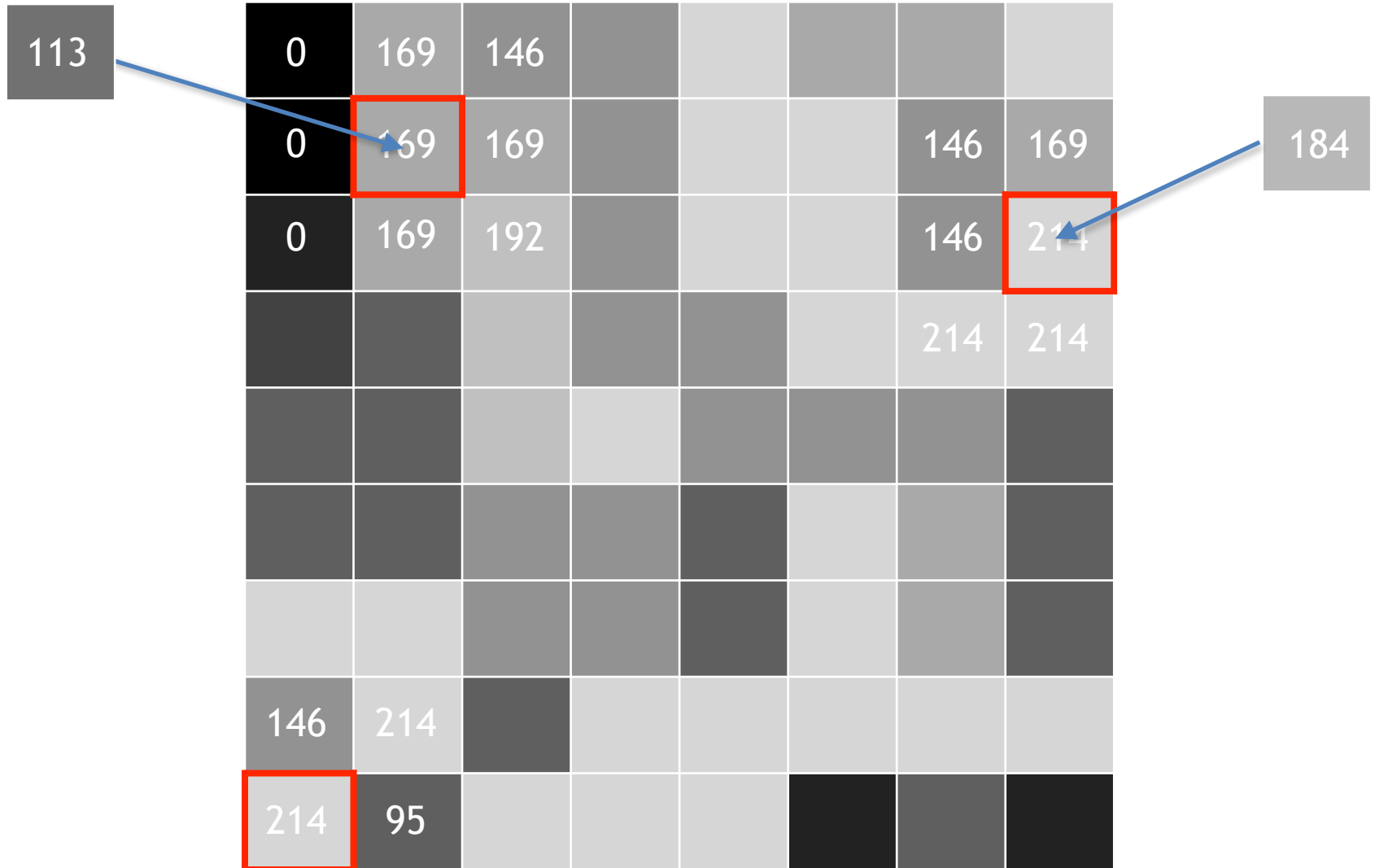
טשטוש תמונה

0	169	146					
0	169	169				146	169
0	169	192				146	214
						214	214
146	214						
214	95						

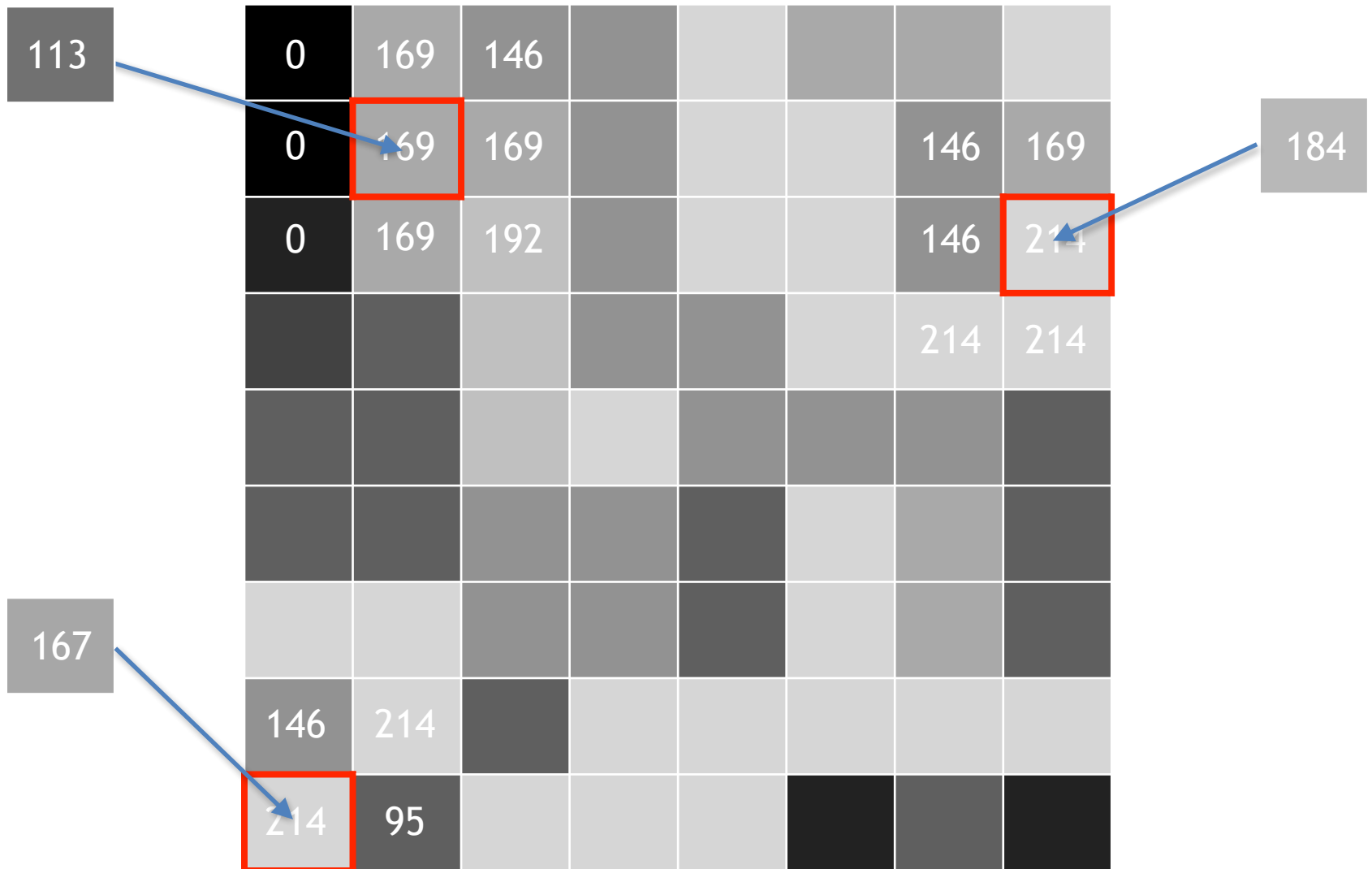
טשטוש תמונה



טשטוש תמונה



טשטוש תמונה



טשטוש תמונה

```
*Python 3.4.1: Untitled*

def average_over_env(pixels, i, j, env):
    """ compute the average pixel value over (i,j)+- env ... """
    s = [0] * 3
    n = 0
    for k in range(-env, env):
        for l in range(-env, env):
            if i+k >= 0 and i+k < len(pixels) and j+l >= 0 and j+l < len(pixels[0]):
                n += 1
                for rgb in range(3):
                    s[rgb] += pixels[i+k][j+l][rgb]
    return tuple(int(s[rgb] / n) for rgb in range(3))

def blur(pixels, env = 1):
    """ returns a blurred image ... """
    new_pixels = [[0]*len(pixels[0])] * len(pixels)
    for i in range(len(pixels)):
        for j in range(len(pixels[0])):
            new_pixels[i][j] = average_over_env(pixels, i, j, env)
    return new_pixels
```

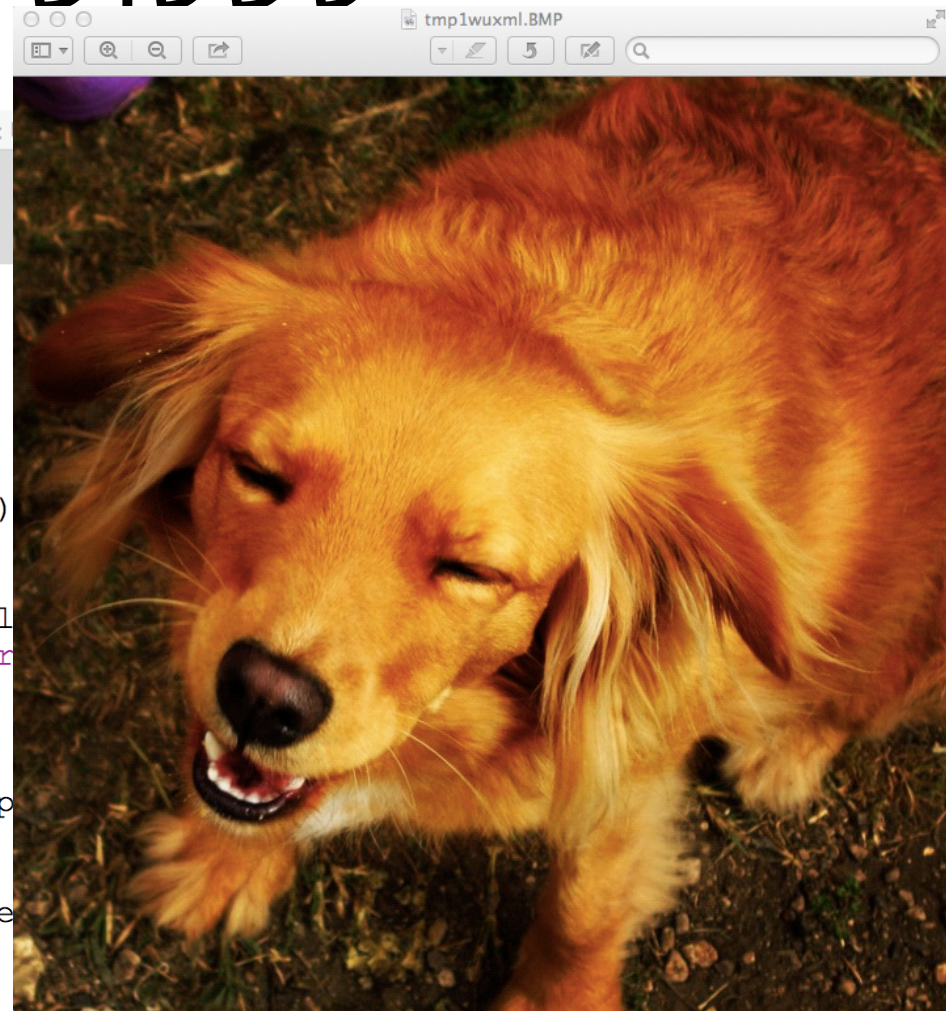
Ln: 43 Col: 0

טשטוש תמונה

```
*Python 3.4.1:

def average_over_env(pixels, i, j, env):
    """ compute the average pixel value over
    s = [0] * 3
    n = 0
    for k in range(-env, env):
        for l in range(-env, env):
            if i+k >= 0 and i+k < len(pixels):
                n += 1
                for rgb in range(3):
                    s[rgb] += pixels[i+k][j+l]
    return tuple(int(s[rgb] / n) for rgb in range(3))

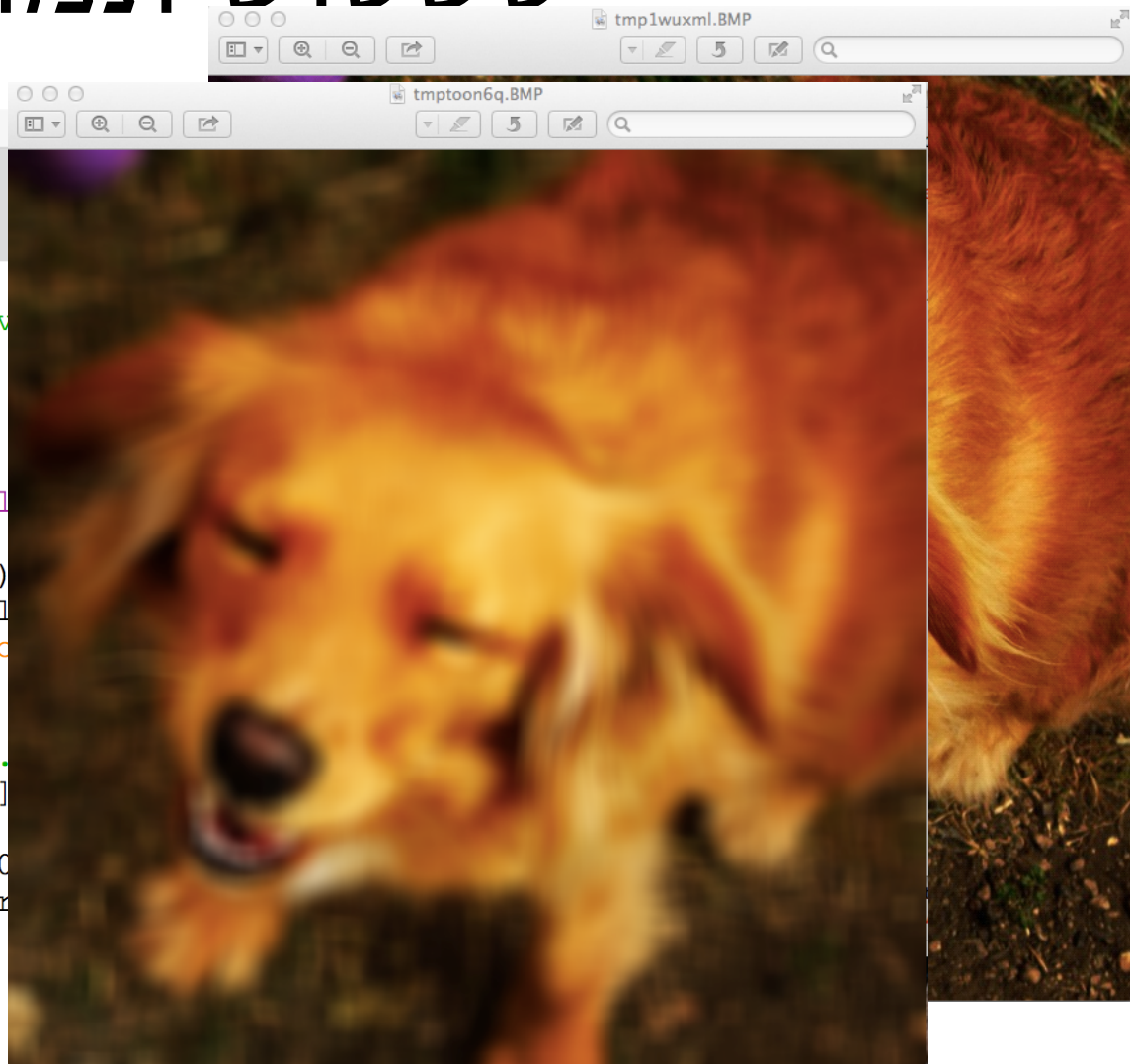
def blur(pixels, env = 1):
    """ returns a blurred image ... """
    new_pixels = [[0]*len(pixels[0])] * len(pixels)
    for i in range(len(pixels)):
        for j in range(len(pixels[0])):
            new_pixels[i][j] = average_over_env(pixels, i, j, env)
    return new_pixels
```



טשטוש תמונה

```
def average_over_env(pixels, i, j, env):
    """ compute the average pixel value over the environment """
    s = [0] * 3
    n = 0
    for k in range(-env, env):
        for l in range(-env, env):
            if i+k >= 0 and i+k < len(pixels) and j+l >= 0 and j+l < len(pixels[0]):
                n += 1
                for rgb in range(3):
                    s[rgb] += pixels[i+k][j+l][rgb]
    return tuple(int(s[rgb] / n) for rgb in range(3))

def blur(pixels, env = 1):
    """ returns a blurred image """
    new_pixels = [[0]*len(pixels[0]) for i in range(len(pixels))]
    for i in range(len(pixels)):
        for j in range(len(pixels[0])):
            new_pixels[i][j] = average_over_env(pixels, i, j, env)
    return new_pixels
```



טשטוש תמונה

